PROYECTO ANDROID / INFORME BASE

AraDirectorio

Usuarios, publicaciones y favoritos sin conexión

Kotlin + Jetpack Compose + Retrofit + Room + MVVM

Asignatura: desarrollo de aplicaciones móviles

Fecha: 23 de octubre de 2026

Estado de esta versión

Implementación fuente preparada para abrir en Android Studio. La compilación y las pruebas en dispositivo están pendientes. Este documento no acredita una ejecución realizada.

Antes de entregar: incorporar las cinco capturas reales solicitadas, registrar los resultados de pruebas y añadir el enlace auténtico del repositorio. No se incluyen capturas simuladas.

1. Objetivo y comportamiento

Desarrollar un directorio que consulte usuarios de JSONPlaceholder, muestre su información y publicaciones y permita conservar favoritos en el dispositivo. El usuario puede navegar entre Directorio, Detalle y Favoritos.

Directorio

Solicita GET /users y muestra nombre, correo y nombre de la empresa. Ofrece actualización manual y acceso al detalle.

Detalle

Muestra contacto, dirección, empresa y ubicación. Consulta GET /posts?userId={id}. El botón Volver y el gesto Atrás regresan a la lista.

Favoritos

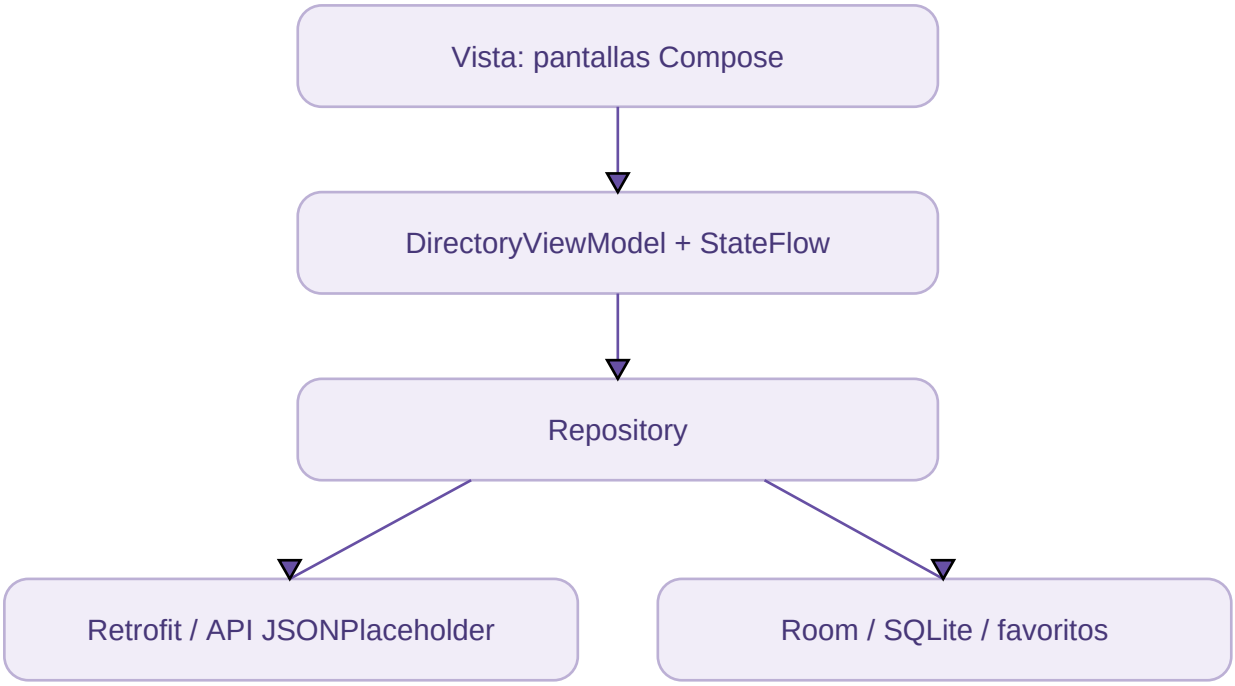
Guarda el usuario completo en Room con su ID como clave primaria. Permite agregar, quitar y leer favoritos sin Internet, incluso después de reiniciar la app.

Estados

Loading muestra un indicador; Success muestra datos o una lista vacía; Error muestra un mensaje y Reintentar. Los fallos de almacenamiento se comunican en una barra de aviso.

Alcance offline: los perfiles favoritos se conservan; las publicaciones necesitan Internet. No se guarda una copia completa del directorio ni un historial de publicaciones.

2. Arquitectura MVVM



Las flechas indican solicitudes desde la vista hacia los datos. Las respuestas y los flujos de estado actualizan la pantalla en sentido inverso.

Vista: representa datos y envía acciones; no accede directamente a Retrofit ni al DAO.

ViewModel: administra estados, selección, cancelación de peticiones y acciones. No conserva referencias a la Activity.

Repositorio: ofrece una entrada común a API y almacenamiento local. **Room:** entidad, DAO y base SQLite. El JSON conserva los campos anidados del perfil en este ejercicio; para consultas más complejas convendría normalizar las columnas.

3. Ciclo de vida y carga

onCreate()

Inicializa la Activity, obtiene el repositorio compartido y compone la interfaz. El proveedor obtiene el ViewModel existente o crea uno nuevo.

Inicialización del ViewModel

Su bloque init observa los favoritos y solicita la lista una vez por instancia. Las corrutinas se ejecutan en viewModelScope.

onStart() y onStop()

collectAsStateWithLifecycle observa los flujos mientras la interfaz está al menos STARTED. La colección se pausa cuando deja de estar activa; no se lanza una petición por cada recomposición.

onResume() y onPause()

No disparan descargas automáticamente. Al regresar, el usuario puede pulsar Actualizar o Reintentar.

Rotación y onDestroy()

La Activity puede recrearse, pero el ViewModel conserva usuarios, selección y publicaciones durante el cambio de configuración. Room conserva favoritos independientemente de la Activity.

onCleared() y muerte de proceso

viewModelScope cancela sus trabajos cuando el ViewModel se elimina definitivamente. El sistema puede terminar el proceso sin llamar onDestroy; por eso los favoritos se escriben al pulsar guardar y no al cerrar la app.

Limitación: tras muerte del proceso se recuperan los favoritos, pero no la pantalla de detalle. Tras una rotación sí se conserva la selección. La navegación usa un estado de selección y BackHandler, sin una segunda Activity.

4. Errores, concurrencia y persistencia

NetworkStatus verifica si la red activa tiene capacidad VALIDATED antes de cada carga. Esta comprobación es orientativa: la conexión puede cambiar después; por eso también se capturan IOException y HttpException.

Las peticiones tienen un límite de 25 segundos. Un nuevo intento cancela la petición anterior; seleccionar otro usuario cancela la descarga previa de publicaciones. CancellationException se propaga para respetar la cancelación de corrutinas.

Room permite consultas suspendidas sin bloquear la interfaz. Su Flow actualiza las estrellas o botones al insertar o borrar. Durante una escritura se deshabilita el botón del usuario para evitar pulsaciones duplicadas.

No se realizan operaciones de red para leer favoritos. Desinstalar la app o borrar sus datos elimina la base local. No existe sincronización de favoritos entre dispositivos.

Relación con la Tarea 4

No se recibió el código de la Tarea 4. Se implementó persistencia con Room. Para acreditar la reutilización literal, debe compararse e integrarse el almacenamiento utilizado en aquella tarea antes de la entrega.

Fuentes de consulta

https://jsonplaceholder.typicode.com/

https://developer.android.com/topic/libraries/architecture/viewmodel

https://developer.android.com/training/data-storage/room

https://square.github.io/retrofit/

5. Evidencias pendientes

Inserta capturas reales después de ejecutar el proyecto. Las siguientes instrucciones son un plan de verificación, no resultados observados.

01 / Carga

En el emulador, configura conexión lenta si está disponible; pulsa Actualizar y captura el indicador Cargando información.

02 / Éxito

Con conexión, captura la lista con nombre, correo y empresa. Abre varios usuarios y comprueba que cada lista de publicaciones pertenece al ID seleccionado.

03 / Error o sin conexión

Desactiva Wi-Fi y datos del emulador. Pulsa Actualizar y captura el aviso y Reintentar. Reconecta y verifica la recuperación.

04 / Detalle

Abre un usuario con conexión y captura perfil y publicaciones; puedes necesitar dos imágenes por el desplazamiento.

05 / Favoritos offline

Guarda dos usuarios, cierra la app, desactiva Internet y vuelve a abrirla. En Favoritos, captura la lista. Abre un perfil para verificar que los datos siguen disponibles.

Pruebas adicionales: quitar un favorito offline y reiniciar; girar el teléfono durante la carga; comprobar Atrás del sistema; probar lista vacía; simular un error HTTP con un proxy de pruebas. Registrar resultado esperado y resultado obtenido en la versión final.

6. Conclusiones y entrega

La implementación separa la interfaz de las consultas y del almacenamiento. Retrofit representa los endpoints con funciones suspendidas y Room permite conservar perfiles favoritos sin depender de Internet. Los estados explícitos permiten comunicar una carga o un fallo y ofrecer recuperación mediante reintentos.

ViewModel conserva el estado durante cambios de configuración. La persistencia duradera corresponde a Room; mantener una lista únicamente en memoria no bastaría para recuperarla después del cierre del proceso.

Estas conclusiones describen el diseño del código. El cumplimiento práctico debe confirmarse compilando y ejecutando las pruebas en Android Studio. No se han verificado en un dispositivo desde este entorno.

Repositorio

Pendiente de publicación. Sigue los comandos de README.md, sustituye TU_USUARIO y agrega aquí la URL real del repositorio. No se ha creado un enlace ficticio.

Lista de revisión antes de entregar

• Compilar y ejecutar en Android Studio con JDK 17 y SDK 35.

• Confirmar favoritos después de cerrar y abrir sin conexión.

• Añadir capturas reales de carga, éxito, desconexión, detalle y favoritos.

• Integrar la persistencia de la Tarea 4 si difiere de Room.

• Publicar el código y reemplazar la nota de repositorio pendiente.

Código / Models.kt

```
package com.aranza.directorio

data class Geo(val lat: String, val lng: String)
data class Address(val street: String, val suite: String, val city: String,
                   val zipcode: String, val geo: Geo)
data class Company(val name: String, val catchPhrase: String, val bs: String)
data class User(val id: Int, val name: String, val username: String,
                val email: String, val phone: String, val website: String,
                val address: Address, val company: Company)
data class Post(val userId: Int, val id: Int, val title: String, val body: String)

sealed interface LoadState<out T> {
    data object Loading : LoadState<Nothing>
    data class Success<T>(val data: T) : LoadState<T>
    data class Error(val message: String) : LoadState<Nothing>
}
```

Código / Data.kt

```
package com.aranza.directorio

import android.content.Context
import android.net.ConnectivityManager
import android.net.NetworkCapabilities
import androidx.room.*
import com.google.gson.Gson
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.map
import okhttp3.OkHttpClient
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import retrofit2.http.GET
import retrofit2.http.Query
import java.io.IOException
import java.util.concurrent.TimeUnit

interface ApiService {
    @GET("users") suspend fun users(): List<User>
    @GET("posts") suspend fun posts(@Query("userId") userId: Int): List<Post>
}

// El JSON conserva el usuario completo: empresa, direccion y contacto.
@Entity(tableName = "favorites")
data class FavoriteEntity(@PrimaryKey val id: Int, val userJson: String)
@Dao
interface FavoriteDao {
    @Query("SELECT * FROM favorites ORDER BY id")
    fun observe(): Flow<List<FavoriteEntity>>
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun save(value: FavoriteEntity)
    @Query("DELETE FROM favorites WHERE id = :id") suspend fun delete(id: Int)
}
@Database(entities = [FavoriteEntity::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() { abstract fun favorites(): FavoriteDao }

class Repository(private val api: ApiService, private val dao: FavoriteDao) {
    private val gson = Gson()
    val favorites: Flow<List<User>> = dao.observe().map { rows ->
        rows.map { gson.fromJson(it.userJson, User::class.java) }
    }
    suspend fun users() = api.users()
    suspend fun posts(id: Int) = api.posts(id)
    suspend fun save(user: User) = dao.save(FavoriteEntity(user.id, gson.toJson(user)))
    suspend fun delete(id: Int) = dao.delete(id)
}

class NoInternet : IOException()
class NetworkStatus(context: Context) {
    private val manager = context.getSystemService(ConnectivityManager::class.java)
    fun online(): Boolean {
        val caps = manager.getNetworkCapabilities(manager.activeNetwork) ?: return false
        return caps.hasCapability(NetworkCapabilities.NET_CAPABILITY_VALIDATED)
    }
}

object Graph {
    @Volatile private var instance: Repository? = null
    fun repository(context: Context): Repository = instance ?: synchronized(this) {
        instance ?: run {
            val db = Room.databaseBuilder(context.applicationContext,
                AppDatabase::class.java, "directorio.db").build()
            val client = OkHttpClient.Builder()
                .callTimeout(25, TimeUnit.SECONDS).build()
            val api = Retrofit.Builder()
                .baseUrl("https://jsonplaceholder.typicode.com/")
                .client(client).addConverterFactory(GsonConverterFactory.create())
                .build().create(ApiService::class.java)
            Repository(api, db.favorites()).also { instance = it }
        }
    }
}
```

Código / DirectoryViewModel.kt

```
package com.aranza.directorio

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*
import retrofit2.HttpException
import java.io.IOException

class DirectoryViewModel(private val repo: Repository,
                         private val network: NetworkStatus) : ViewModel() {
    private val _users = MutableStateFlow<LoadState<List<User>>>(LoadState.Loading)
    val users = _users.asStateFlow()
    private val _posts = MutableStateFlow<LoadState<List<Post>>>(LoadState.Loading)
    val posts = _posts.asStateFlow()
    private val _favorites = MutableStateFlow<List<User>>(emptyList())
    val favorites = _favorites.asStateFlow()
    private val _notice = MutableStateFlow<String?>(null)
    val notice = _notice.asStateFlow()
    private val _selected = MutableStateFlow<User?>(null)
    val selected = _selected.asStateFlow()
    private val _saving = MutableStateFlow<Set<Int>>(emptySet())
    val saving = _saving.asStateFlow()
    private var usersJob: Job? = null
    private var postsJob: Job? = null

    init {
        viewModelScope.launch {
            repo.favorites.catch { _notice.value = "No se pudieron leer los favoritos." }
                .collect { _favorites.value = it }
        }
        refresh()
    }
    fun clearNotice() { _notice.value = null }
    private fun message(e: Exception): String = when (e) {
        is NoInternet -> "Sin conexión a Internet. Puedes consultar tus favoritos."
        is HttpException -> "El servidor respondió con error ${e.code()}. Reintenta."
        is IOException -> "No se pudo conectar. Comprueba Internet y reintenta."
        else -> "No se pudieron procesar los datos. Inténtalo otra vez."
```

Código / DirectoryViewModel.kt / continuación

```
    }
    fun refresh() {
        usersJob?.cancel()
        usersJob = viewModelScope.launch {
            _users.value = LoadState.Loading
            try {
                if (!network.online()) throw NoInternet()
                _users.value = LoadState.Success(repo.users())
            } catch (e: CancellationException) { throw e
            } catch (e: Exception) { _users.value = LoadState.Error(message(e)) }
        }
    }
    fun select(user: User) { _selected.value = user; loadPosts() }
    fun back() { postsJob?.cancel(); _selected.value = null }
    fun loadPosts() {
        val id = _selected.value?.id ?: return
        postsJob?.cancel()
        postsJob = viewModelScope.launch {
            _posts.value = LoadState.Loading
            try {
                if (!network.online()) throw NoInternet()
                _posts.value = LoadState.Success(repo.posts(id))
            } catch (e: CancellationException) { throw e
            } catch (e: Exception) { _posts.value = LoadState.Error(message(e)) }
        }
    }
    fun toggle(user: User) {
        if (user.id in _saving.value) return
        val remove = _favorites.value.any { it.id == user.id }
        _saving.value = _saving.value + user.id
        viewModelScope.launch {
            try {
                if (remove) repo.delete(user.id) else repo.save(user)
            } catch (e: CancellationException) { throw e
            } catch (e: Exception) { _notice.value = "No se pudo guardar el cambio de favorito."
            } finally { _saving.value = _saving.value - user.id }
        }
    }
}
```

Código / MainActivity.kt

```
package com.aranza.directorio

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.BackHandler
import androidx.activity.compose.setContent
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import androidx.lifecycle.viewmodel.compose.viewModel

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val repository = Graph.repository(applicationContext)
        val network = NetworkStatus(applicationContext)
        setContent {
            val vm: DirectoryViewModel = viewModel(factory = object : ViewModelProvider.Factory
{
                @Suppress("UNCHECKED_CAST")
                override fun <T : ViewModel> create(modelClass: Class<T>): T =
                    DirectoryViewModel(repository, network) as T
            })
            MaterialTheme(colorScheme = lightColorScheme(
                primary = Color(0xFF6750A4), background = Color(0xFFF8F6FC))) {
                DirectoryApp(vm)
            }
        }
    }
}

@Composable
fun DirectoryApp(vm: DirectoryViewModel) {
    val users by vm.users.collectAsStateWithLifecycle()
    val posts by vm.posts.collectAsStateWithLifecycle()
    val favorites by vm.favorites.collectAsStateWithLifecycle()
    val selected by vm.selected.collectAsStateWithLifecycle()
    val notice by vm.notice.collectAsStateWithLifecycle()
    val saving by vm.saving.collectAsStateWithLifecycle()
    var favoritesTab by rememberSaveable { mutableStateOf(false) }
    val snackbar = remember { SnackbarHostState() }
    LaunchedEffect(notice) {
        notice?.let { snackbar.showSnackbar(it); vm.clearNotice() }
    }
    BackHandler(enabled = selected != null) { vm.back() }
    Scaffold(snackbarHost = { SnackbarHost(snackbar) }) { padding ->
        Column(Modifier.fillMaxSize().padding(padding).padding(horizontal = 20.dp)) {
            Spacer(Modifier.height(16.dp))
```

Código / MainActivity.kt / continuación

```
            Text("AraDirectorio", style = MaterialTheme.typography.headlineLarge)
            Text("Personas y conexiones", color = MaterialTheme.colorScheme.primary)
            Spacer(Modifier.height(12.dp))
            if (selected == null) {
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    FilterChip(selected = !favoritesTab, onClick = { favoritesTab = false },
                        label = { Text("Directorio") })
                    FilterChip(selected = favoritesTab, onClick = { favoritesTab = true },
                        label = { Text("Favoritos (${favorites.size})") })
                }
                if (favoritesTab) {
                    Text("Guardados en este dispositivo. Disponibles sin Internet.")
                    UserList(favorites, favorites, saving, vm::select, vm::toggle)
                } else {
                    TextButton(onClick = vm::refresh) { Text("Actualizar directorio") }
                    when (val state = users) {
                        LoadState.Loading -> Loading()
                        is LoadState.Error -> ErrorPanel(state.message, vm::refresh)
                        is LoadState.Success -> UserList(state.data, favorites, saving,
                            vm::select, vm::toggle)
                    }
                }
            } else {
                val user = selected!!
                TextButton(onClick = vm::back) { Text("Volver") }
                LazyColumn(verticalArrangement = Arrangement.spacedBy(12.dp)) {
                    item {
                        Card(Modifier.fillMaxWidth()) {
                            Column(Modifier.padding(16.dp), verticalArrangement =
Arrangement.spacedBy(6.dp)) {
                                Text(user.name, style = MaterialTheme.typography.titleLarge)
                                Text("Usuario: ${user.username}")
                                Text("Correo: ${user.email}")
                                Text("Teléfono: ${user.phone}")
                                Text("Sitio web: ${user.website}")
                                Text("Empresa: ${user.company.name}")
                                Text(user.company.catchPhrase)
                                Text("Actividad: ${user.company.bs}")
                                Text("Dirección: ${user.address.street}, ${user.address.suite},
${user.address.city}")
                                Text("Código postal: ${user.address.zipcode}")
                                Text("Ubicación: ${user.address.geo.lat},
${user.address.geo.lng}")
                                TextButton(enabled = user.id !in saving, onClick = {
vm.toggle(user) }) {
                                    Text(if (favorites.any { it.id == user.id }) "Quitar de
favoritos" else "Guardar favorito")
                                }
                            }
                        }
                    }
                    item { Text("Publicaciones", style = MaterialTheme.typography.titleLarge) }
                    when (val state = posts) {
                        LoadState.Loading -> item { Loading() }
                        is LoadState.Error -> item { ErrorPanel(state.message +
                            " Las publicaciones necesitan conexión; el perfil guardado sigue
disponible.", vm::loadPosts) }
                        is LoadState.Success -> {
```

Código / MainActivity.kt / continuación

```
                            if (state.data.isEmpty()) item { Text("Este usuario no tiene
publicaciones.") }
                            items(state.data, key = { it.id }) { post ->
                                Card(Modifier.fillMaxWidth()) {
                                    Column(Modifier.padding(16.dp)) {
                                        Text(post.title, style =
MaterialTheme.typography.titleMedium)
                                        Spacer(Modifier.height(8.dp))
                                        Text(post.body)
                                    }
                                }
                            }
                        }
                    }
                    item { Spacer(Modifier.height(16.dp)) }
                }
            }
        }
    }
}

@Composable
fun UserList(users: List<User>, favorites: List<User>, saving: Set<Int>,
             select: (User) -> Unit, toggle: (User) -> Unit) {
    if (users.isEmpty()) Text("No hay usuarios para mostrar.")
    LazyColumn(verticalArrangement = Arrangement.spacedBy(10.dp),
        contentPadding = PaddingValues(vertical = 12.dp)) {
        items(users, key = { it.id }) { user ->
            Card(Modifier.fillMaxWidth().clickable { select(user) }) {
                Column(Modifier.padding(16.dp)) {
                    Text(user.name, style = MaterialTheme.typography.titleMedium)
                    Text(user.email)
                    Text(user.company.name, color = MaterialTheme.colorScheme.primary)
                    TextButton(enabled = user.id !in saving, onClick = { toggle(user) }) {
                        Text(if (favorites.any { it.id == user.id }) "Quitar de favoritos" else
"Guardar favorito")
                    }
                }
            }
        }
    }
}
@Composable
fun Loading() {
    Column(Modifier.fillMaxWidth().padding(24.dp), verticalArrangement =
Arrangement.spacedBy(12.dp)) {
        CircularProgressIndicator()
        Text("Cargando información...")
    }
}
@Composable
fun ErrorPanel(message: String, retry: () -> Unit) {
    Column(Modifier.padding(vertical = 16.dp)) {
        Text(message, color = MaterialTheme.colorScheme.error)
        Button(onClick = retry) { Text("Reintentar") }
    }
}
```

Configuración / permisos y dependencias

app/src/main/AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
    <application android:label="AraDirectorio"
android:theme="@android:style/Theme.Material.Light.NoActionBar" android:allowBackup="false"
android:supportsRtl="true">
        <activity android:name=".MainActivity" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

app/build.gradle.kts

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("org.jetbrains.kotlin.plugin.compose")
    id("com.google.devtools.ksp")
}
android {
    namespace = "com.aranza.directorio"
    compileSdk = 35
    defaultConfig {
        applicationId = "com.aranza.directorio"
        minSdk = 24
        targetSdk = 35
        versionCode = 1
        versionName = "1.0"
    }
    buildFeatures { compose = true }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    kotlinOptions { jvmTarget = "17" }
}
dependencies {
    implementation(platform("androidx.compose:compose-bom:2024.12.01"))
    implementation("androidx.activity:activity-compose:1.9.3")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.foundation:foundation")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.7")
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.7")
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    ksp("androidx.room:room-compiler:2.6.1")
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("com.squareup.retrofit2:converter-gson:2.11.0")
}
```